Landkarten aus Geodaten mit Ti*k*Z zeichnen

Fortbildungsmaterial für Lehrkräfte

11. November 2026

Stumme Karten zum Beschriften, eine Lagekarte für ein Projekt in Sachsen, Kartenprojektionen im Vergleich oder ein Algorithmus, der Umrisse vereinfacht: Karten sind in Geografie, Geschichte, Gemeinschaftskunde, Mathematik und Informatik nützlich. Eine Karte „aus dem Kopf“ mit Ti*k*Z-Koordinaten zu zeichnen, gelingt aber nie überzeugend. Diese Handreichung zeigt einen anderen Weg: Freie Geodaten werden mit einem kleinen Python-Skript in Ti*k*Z-Pfade umgerechnet, danach lässt sich jedes Bundesland, jeder Fluss und jede Stadt einzeln gestalten.

1 Der Arbeitsablauf

Eine Karte entsteht in drei Schritten:

1. **Daten holen.** Freie Geodaten im Format GeoJSON herunterladen, zum Beispiel Bundesländer, Flüsse und Seen.
2. **Umrechnen.** Das Skript `geo2tikz.py` aus dem Ordner `scripts` des Skills projiziert die Daten aufs Papier, vereinfacht die Umrisse, schneidet den Ausschnitt zu und schreibt eine `.tex`-Datei.
3. **Gestalten.** Die erzeugte Datei wird mit `\input` geladen. Farben, Linien und Beschriftungen legt man wie gewohnt in Ti*k*Z fest.

Das Skript braucht Python 3 und zwei Zusatzpakete, die man einmalig installiert:

```
pip install shapely pyproj
```

Für die Karte selbst genügt Ti*k*Z und das ist in der universalen `header.tex` bereits geladen.

2 Geodaten besorgen

2.1 Datenquellen und Lizenzen

Quelle	Inhalt	Lizenz
Natural Earth	Staaten, Bundesländer und Provinzen, Flüsse, Seen, Gebirgsregionen, Straßen, Bahnlinien, Städte	gemeinfrei; Quellenangabe ist trotzdem üblich
historical-basemaps	Weltgrenzen zu Stichjahren, etwa 1938 oder 1945	GPL-3.0: Quelle nennen, abgeleitete Daten bleiben GPL
AWMC geodata	Antike, zum Beispiel das Römische Reich	ODbL: Quelle nennen

Alle drei Quellen liegen auf GitHub: `nvkelso/natural-earth-vector` (Ordner `geojson`), `aourednik/historical-basemaps` und `AWMC/geodata`. Die Rohdaten sind teilweise sehr groß (die Bundesländer der ganzen Welt umfassen 39 MB). Man legt sie deshalb nicht in Materialsammlungen ab, sondern notiert den Download-Befehl. Auf jede Karte gehört eine Quellenzeile.

2.2 Herunterladen

Im Terminal lädt `curl` die Dateien direkt herunter. Für eine Deutschlandkarte mit Bundesländern, Nachbarstaaten, Flüssen, Seen und Städten:

```
B=https://raw.githubusercontent.com/nvkelso/natural-earth-vector/master/geojson
curl -sSfLO $B/ne_10m_admin_1_states_provinces.geojson
curl -sSfLO $B/ne_50m_admin_0_countries.geojson
curl -sSfLO $B/ne_10m_rivers_lake_centerlines.geojson
curl -sSfLO $B/ne_10m_lakes.geojson
curl -sSfLO $B/ne_10m_populated_places_simple.geojson
```

Die Zahl im Dateinamen gibt die Genauigkeit an: `10m` ist der feinste Maßstab (1 : 10 Mio.), `50m` und `110m` sind gröber und reichen für Übersichtskarten von Europa oder der Welt.

3 Umrechnen mit `geo2tikz.py`

3.1 Der Aufruf

Das Skript bekommt den Kartenausschnitt in Grad, die gewünschte Breite in Zentimetern, eine Projektion und eine oder mehrere Ebenen. Alle Ebenen einer Karte werden in **einem** Aufruf umgerechnet, sonst haben sie verschiedene Maßstäbe und die Flüsse liegen neben den Ländern. Mit diesem Aufruf entstand die Datei für die Beispiele dieser Handreichung:

```
python3 geo2tikz.py --bbox 5.8,47.2,15.1,55.1 --breite 9 --proj EPSG:3035 --toleranz 0.03 \
  --ebene "name=laender; datei=ne_10m_admin_1_states_provinces.geojson; filter=iso_a2=DE; id=iso
      _3166_2" \
  --ebene "name=nachbarn; datei=ne_50m_admin_0_countries.geojson; filter=ISO_A2_EH=PL|CZ|AT|CH|FR
      |BE|NL|LU|DK; id=ISO_A2_EH" \
  --ebene "name=fluesse; datei=ne_10m_rivers_lake_centerlines.geojson; id=name" \
  --ebene "name=seen; datei=ne_10m_lakes.geojson; id=name" \
  --ebene "name=staedte; datei=ne_10m_populated_places_simple.geojson; filter=adm0_a3=DEU; id=
      name" \
  -o beispiel_deutschland.tex
```

Das Skript meldet danach, wie viele Objekte jede Ebene enthält und wie groß die Karte wird, hier $9{,}0 \times 11{,}4$ cm.

3.2 Die Optionen

Option	Bedeutung
`--bbox`	Ausschnitt als Länge/Breite links unten und rechts oben in Grad. Bei negativen Werten mit Gleichheitszeichen schreiben: `--bbox=-180,-58,180,84`
`--breite`	Kartenbreite in cm; die Höhe ergibt sich aus der Projektion
`--proj`	`plattkarte` (Standard), `mercator` oder ein EPSG-Code: `EPSG:3035` für Europa (flächentreu), `EPSG:25833` für Ostdeutschland, `EPSG:8857` (Equal Earth) für die Welt
`--toleranz`	Vereinfachung der Umrisse in cm (Kapitel 6); `0` schaltet sie aus
`--ebene`	eine Ebene: `name` (nur Buchstaben), `datei`, optional `filter` und `id`
`-o`	Name der erzeugten Datei

Zwei Fallstricke bei den Filtern: Bundesländer haben in Natural Earth die Kennung `iso_3166_2` (`DE-SN`, `DE-BY` …). Bei Staaten fehlen Frankreich und Norwegen, wenn man nach `ISO_A2` filtert,

weil dort `-99` eingetragen ist. Deshalb filtert man Staaten nach `ISO_A2_EH` oder `ADM0_A3` (`DEU`, `FRA`).

3.3 Was in der erzeugten Datei steht

Für jede Ebene schreibt das Skript einen Befehl, der alle Objekte zeichnet, und einen leeren Stil für die Ebene und für jedes einzelne Objekt. Außerdem legt es für jedes Objekt einen Punkt an, der garantiert im Inneren liegt. Gekürzt sieht das so aus:

```
\tikzset{geo laender/.style={},
  geo laender/DE-SN/.style={},
  geo laender/DE-BY/.style={}, ...}
\newcommand\geolaender{%
\path[geo laender, geo laender/DE-SN, even odd rule] (8.08,5.57) -- (7.90,5.56) -- ... -- cycle;
...
\coordinate (laender-DE-SN) at (7.17,5.32);
...}
```

Daraus ergibt sich die Arbeitsweise:

Element	**Beispiel**	**Zweck**
Befehl der Ebene	`\geolaender`	zeichnet alle Objekte der Ebene
Stil der Ebene	`geo laender/.style={...}`	Aussehen aller Objekte
Stil eines Objekts	`geo laender/DE-SN/.style={...}`	ein Objekt hervorheben
Punkt im Objekt	`(laender-DE-SN)`	Beschriftung, Markierung

Weil alle Stile zunächst leer sind, zeichnet ein Befehl wie `\geolaender` ohne eigenen Stil nichts. Erst ein Stil mit `fill` oder `draw` macht die Ebene sichtbar.

4 Die erste Karte

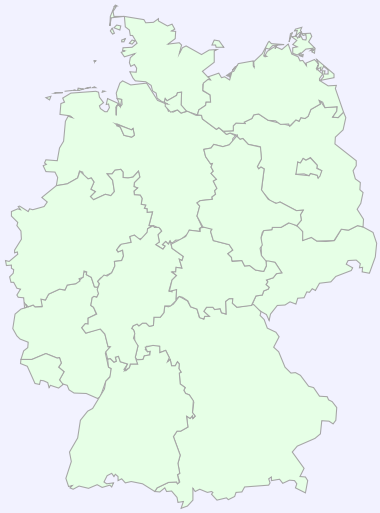
Die erzeugte Datei wird in der Präambel geladen:

```
\input{beispiel_deutschland.tex}
```

Danach genügt ein Stil für die Ebene und der Befehl `\geolaender`. Mit `scale` lässt sich die Karte verkleinern:

```
\begin{tikzpicture}[scale=0.6,
  geo laender/.style={fill=green!10, draw=gray!70, line width=0.4pt}]
  \geolaender
\end{tikzpicture}
```

Ausgabe



Die Stile stehen hier in den Optionen der `tikzpicture`. Sie gelten dann nur für diese eine Karte, und dieselben Daten lassen sich im selben Dokument für eine zweite Karte mit anderem Aussehen verwenden.

5 Karten gestalten

5.1 Mehrere Ebenen, Hervorhebungen und Städte

Die Ebenen werden von unten nach oben gezeichnet: erst die Nachbarstaaten, dann die Bundesländer, darüber Seen und Flüsse und zuletzt die Beschriftung. `\clip` schneidet alles am Kartenrand ab; die Maße stehen in der letzten Zeile der erzeugten Datei. Die Ebene `staedte` zeichnet selbst nichts, legt aber die Punkte der Städte an.

```
\begin{tikzpicture}[scale=0.85,
  geo nachbarn/.style={fill=gray!12, draw=gray!50, line width=0.3pt},
  geo laender/.style={fill=green!8, draw=gray!70, line width=0.4pt},
  geo laender/DE-SN/.style={fill=orange!45},
  geo seen/.style={fill=blue!25, draw=blue!50, line width=0.3pt},
  geo fluesse/.style={draw=blue!60, line width=0.6pt, line cap=round, line join=round}]
  \clip (0,0) rectangle (9,11.37);
  \geonachbarn \geolaender \geoseen \geofluesse \geostaedte
  \foreach \s/\n/\pos in {Berlin/Berlin/right, Hamburg/Hamburg/left, Leipzig/Leipzig/left,
                          Dresden/Dresden/right, Munich/München/right, Cologne/Köln/left}
    \fill (staedte-\s) circle (1.5pt) node[\pos, font=\scriptsize] {\n};
  \node[below right, font=\scriptsize\itshape, text=blue!70!black] at (fluesse-Elbe) {Elbe};
  \node[anchor=south east, font=\tiny, fill=white, inner sep=1pt] at (9,0) {Kartendaten: Natural
      Earth (gemeinfrei)};
\end{tikzpicture}
```

Ausgabe



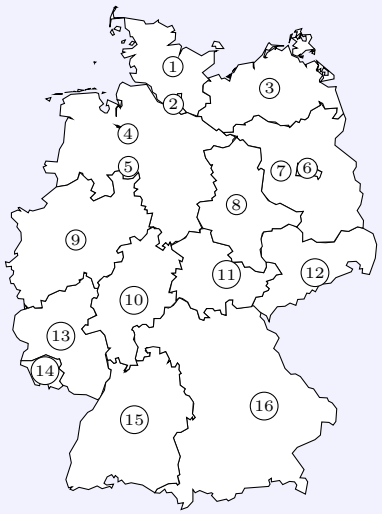
Die Namen der Städte stammen aus Natural Earth und sind englisch (`Munich`, `Cologne`). Umlaute ersetzt das Skript in den Namen der Punkte, aus Düsseldorf wird `staedte-Duesseldorf`. Welche Namen es gibt, zeigt ein Blick in die erzeugte Datei. Bei Flüssen lohnt das besonders: Der Rhein heißt je nach Abschnitt `Rhein`, `Rhin` oder `Rhine`.

5.2 Eine stumme Karte

Für ein Arbeitsblatt bekommen die Bundesländer Nummern statt Namen. Die Schleife setzt an den Punkt jedes Landes einen kleinen Kreis mit seiner Nummer:

```
\begin{tikzpicture}[scale=0.6,
  geo laender/.style={fill=white, draw=black, line width=0.4pt}]
  \geolaender
  \foreach \l [count=\n] in {SH,HH,MV,HB,NI,BE,BB,ST,NW,HE,TH,SN,RP,SL,BW,BY}
    \node[circle, fill=white, draw, inner sep=0.8pt, font=\tiny] at (laender-DE-\l) {\n};
\end{tikzpicture}
```

Ausgabe



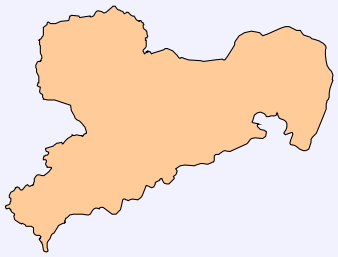
Die Lösung ergibt sich aus der Reihenfolge in der Liste: 1 Schleswig-Holstein, 2 Hamburg, 3 Mecklenburg-Vorpommern, 4 Bremen und so weiter. Wer die Reihenfolge der Liste ändert, bekommt eine neue Fassung für die Parallelklasse.

6 Umrisse vereinfachen

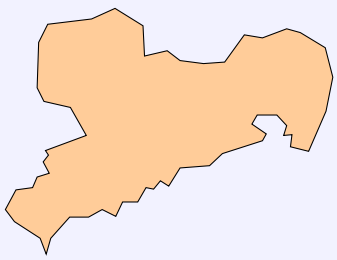
Echte Grenzen haben sehr viele Eckpunkte. Für eine kleine Karte sind die meisten davon unsichtbar, machen die Datei aber groß und LaTeX langsam. Das Skript vereinfacht die Umrisse deshalb mit dem Douglas-Peucker-Algorithmus: Ein Eckpunkt fällt weg, wenn er weniger als `--toleranz` Zentimeter von der Linie zwischen seinen Nachbarn abweicht. Sachsen, dreimal mit verschiedener Toleranz umgerechnet:

```
\begin{tikzpicture}[geo sachsenA/.style={fill=orange!40, draw=black, line width=0.4pt}]
  \geosachsenA \node at (2.3,-0.4) {\small Toleranz 0: 645 Ecken};
\end{tikzpicture}\hfill
\begin{tikzpicture}[geo sachsenB/.style={fill=orange!40, draw=black, line width=0.4pt}]
  \geosachsenB \node at (2.3,-0.4) {\small 0,05\,cm: 56 Ecken};
\end{tikzpicture}\hfill
\begin{tikzpicture}[geo sachsenC/.style={fill=orange!40, draw=black, line width=0.4pt}]
  \geosachsenC \node at (2.3,-0.4) {\small 0,25\,cm: 14 Ecken};
\end{tikzpicture}
```

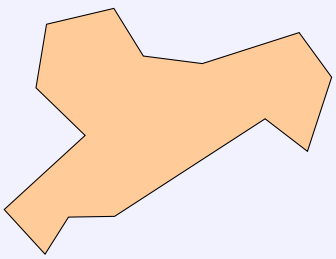
Ausgabe



Toleranz 0: 645 Ecken



0,05 cm: 56 Ecken



0,25 cm: 14 Ecken

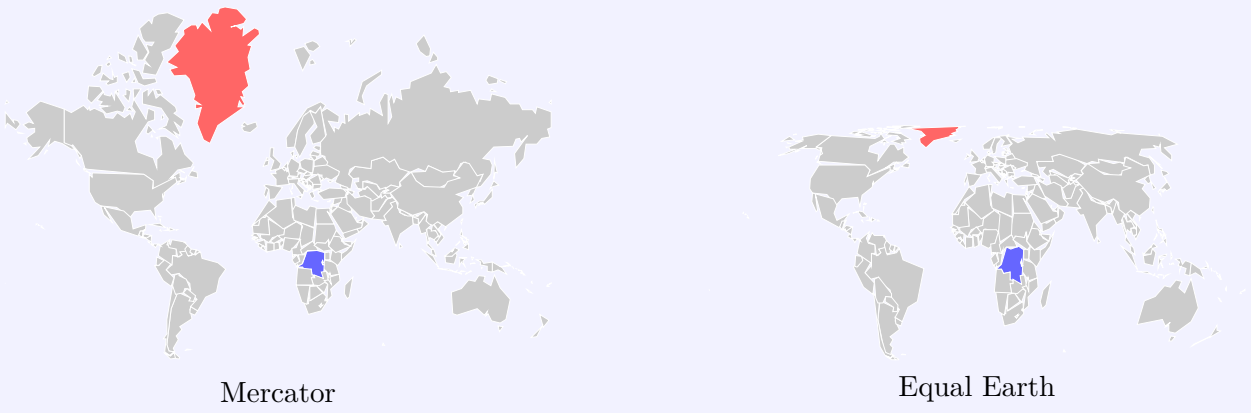
Die Ebenen heißen hier `sachsenA`, `sachsenB` und `sachsenC`, weil jede Datei sonst denselben Befehl `\geosachsen` anlegen würde. Mit dem Standardwert `0.02` sieht man bei normalen Kartengrößen keinen Unterschied zum Original. Bei großer Toleranz können zwischen benachbarten Ländern kleine Lücken entstehen, weil jedes Land für sich vereinfacht wird.

7 Projektionen vergleichen

Die Erde ist eine Kugel, die Karte ist flach. Jede Projektion verzerrt deshalb etwas: Winkel, Flächen oder Entfernungen. Die Mercator-Projektion ist winkeltreu, vergrößert aber Flächen zu den Polen hin sehr stark. Die Projektion *Equal Earth* ist flächentreu. Beide Karten entstanden aus denselben Daten (`ne_110m_admin_0_countries`) mit `--proj mercator` bzw. `--proj EPSG:8857`. Hervorgehoben sind Grönland und die Demokratische Republik Kongo:

```
\begin{tikzpicture}[scale=0.95,
  geo weltmercator/.style={fill=gray!40, draw=white, line width=0.15pt},
  geo weltmercator/GRL/.style={fill=red!60},
  geo weltmercator/COD/.style={fill=blue!60}]
  \geoweltmercator
  \node[font=\small] at (3.8,-0.35) {Mercator};
\end{tikzpicture}\hfill
\begin{tikzpicture}[scale=0.95,
  geo weltequal/.style={fill=gray!40, draw=white, line width=0.15pt},
  geo weltequal/GRL/.style={fill=red!60},
  geo weltequal/COD/.style={fill=blue!60}]
  \geoweltequal
  \node[font=\small] at (3.8,-0.35) {Equal Earth};
\end{tikzpicture}
```

Ausgabe



In der Mercator-Karte wirkt Grönland (rot) größer als der Kongo (blau). Tatsächlich ist der Kongo mit rund 2,34 Mio. km² sogar etwas größer als Grönland mit rund 2,17 Mio. km², wie die flächentreue Karte zeigt. Die Kennungen `GRL` und `COD` sind die dreistelligen Ländercodes aus dem Feld `ADM0_A3`.

8 Historische Karten

Grenzen vergangener Zeiten gibt es ebenfalls als freie Geodaten. *historical-basemaps* enthält Weltkarten zu Stichjahren wie 1938 oder 1945, *AWMC geodata* unter anderem die Ausdehnung des Römischen Reichs. Der Arbeitsablauf ist derselbe: herunterladen, mit `geo2tikz.py` umrechnen, gestalten. Zwei Dinge sind dabei anders als bei Natural Earth: Die Lizenzen (GPL-3.0 bzw. ODbL) verlangen eine Quellenangabe auf der Karte, und historische Grenzen sind oft umstritten oder nur ungefähr bekannt. Darauf sollte das Material hinweisen.

9 Zusammenfassung

Die wichtigsten Techniken im Überblick:

- Karten nicht von Hand zeichnen, sondern freie Geodaten mit `geo2tikz.py` umrechnen.
- Alle Ebenen einer Karte in einem einzigen Aufruf umrechnen, damit sie zusammenpassen.
- Die erzeugte Datei mit `\input` laden; jede Ebene hat einen Befehl (`\geolaender`), einen Stil (`geo laender`) und Stile für jedes Objekt (`geo laender/DE-SN`).
- Ebenen von unten nach oben zeichnen und mit `\clip` am Kartenrand abschneiden.
- Beschriften über die Punkte im Inneren, etwa `(laender-DE-SN)` oder `(staedte-Leipzig)`.
- Projektion nach Zweck wählen: `EPSG:3035` für Europa, `EPSG:25833` für Sachsen, `EPSG:8857` für flächentreue Weltkarten.
- Auf jede Karte eine Quellenzeile setzen.

Die Karten lassen sich fächerübergreifend nutzen: stumme Karten in Geografie und Geschichte, Projektionen und Flächenberechnung in Mathematik (die Fläche des vereinfachten Sachsens lässt sich mit der Gaußschen Trapezformel berechnen), Douglas-Peucker als rekursiver Algorithmus und GeoJSON als Datenformat in Informatik. Eine passende Aufgabe für Fortgeschrittene ist das Färben einer Europakarte mit möglichst wenigen Farben, sodass Nachbarländer nie dieselbe Farbe haben.